第6章　中間コード生成

6.1　概要

構文解析によって，プログラミング言語に対する入力プログラムの構文的な正しさをチェックするとともに，入力プログラムの意味を構文解析木として与える．構文解析木に沿って，プログラムの動作を実現する命令の系列を生成する．開発者が用いるソース言語の抽象度が高いほど，プログラムは構造的に整理されて理解が容易な形で記述される．これに対して計算機の実行命令は直線的な系列であり，計算機固有のアーキテクチャの影響を受ける．近代のコンパイラでは，命令コードは複数の段階を経て生成されるのが一般的である．まず，単純な制御構造とメモリアクセスを直線的に表現できる中間コードを機械的に生成し，その後，実際のアーキテクチャに沿ったよりハードウェアに近い命令に変換する．

中間コードを一旦生成することで，アーキテクチャに依存しない，入力プログラムが持つ制御構造，メモリアクセスを抽出して，入力プログラムの性質に基づいて最適化することができる．さらに中間コードで用いる変数を数の制限の大きいレジスタに配分して，プログラムに対してメモリ階層の観点から効率的に実行する割り当て方法を実現することができる．

6.1.1　抽象構文と抽象構文木

抽象構文は，プログラムを定義する文脈自由文法が表現する文法のうち，プログラムの振舞いに関連する部分のみを抽象した文法によって定義される．例えば，四則演算に関する構文は，図 6.1 で十分である．この文脈自由文法は曖昧であるが，四則演算の演算子が $+$, $-$, $*$, / であり，それらが変数または数から再帰的に構成される式を 2 つ両側に持つ演算であることを表している．一旦，式の構文解析木が構成されれば，その意味（値）を計算するには，抽象構文が表現する意味に沿って計算する命令系列を生成すればよい．

抽象構文木 (abstract syntax tree) は，本質的に意味を表す抽象構文に沿って構成される木構造である．コード生成において，構文解析木に現れる非終端記号の情報は必要でない場合が多く，生成規則の持つ意味をノードに記述した方がコード生成には都合が良い．木構造がコードの意味を直接的に表現していると考えられることから，抽象構文木が中間コードであるとみなすことができる．例えば，代入式 `w=x+y*z` の抽象構文木は図 6.2 のようになる．

実際には構文木の構成において，x,y,z はそれぞれトークンとして ID として扱われるが，実際のコード生成においては，変数名を必要とする．字句解析においてもトークンの他に実際の識別子の記号列が保存されている．(lex における `yytext` など)

$$
\begin{aligned}
E &\rightarrow E + E \\
E &\rightarrow E - E \\
E &\rightarrow E * E \\
E &\rightarrow E \ / \ E \\
E &\rightarrow \texttt{ID} \\
E &\rightarrow \texttt{NUM} \\
S &\rightarrow \texttt{ID} = E
\end{aligned}
$$

図 6.1: 四則演算式と代入文の抽象構文

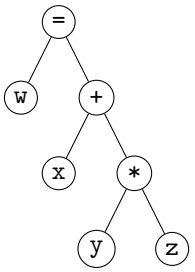


図 6.2: 抽象構文木の例

6.1.2　逆ポーランド記法

逆ポーランド記法は引数の数が固定の演算を後置形式で記載する．計算に用いる値をメモリの値を push 命令で参照することによって計算することができ，算術式の中間コードとして古くから用いられている．逆ポーランド記法からより低レベルの機械語（バイナリコード）へ変換することは容易である．演算の引数となる変数を ID で参照するとき，push ID でその値をスタックに積む，演算を適用するとき，n 引数の演算では，n 個の値をスタックから pop して演算を適用し，その結果をスタックに push する．例えば，x+y*z という入力に対しては，push x, push y, push z, $*$, $+$ という命令列によってスタックの先頭に値の計算が積まれる (図 6.3).

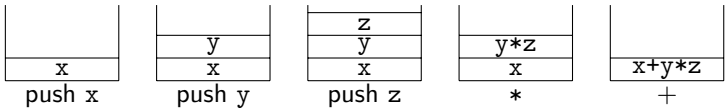


図 6.3: 逆ポーランド記法の計算

6.1.3　3 番地コード

3 番地コード (3-address code) は，命令を一般的に $C = A\theta B$ の形に限定して動作を記述する．ここで，A,B,C はメモリ番地であり，A のメモリの内容と B のメモリの内容に θ の演算を適用して，結果を C の番地に格納することを表す．これ以外に単項演算，条件付きジャンプ命令などを

	一般形	演算子 θ
代入文	$C = A\theta B$ $B = \theta A$ B=A	+,-,*,/, and,or,not >,>=,=,!=,<=,<,-
ジャンプ文	if A goto L if-not A goto L goto L	

図 6.4: 3 番地コード

用いることによって一般的な計算を単純な命令の組み合わせで記述することができる．各命令にはラベル L をつけることができ，ジャンプ命令の飛び先とすることができる．3 番地コードを機械語に変換することは容易であり，3 番地コードによって最適化を行うことでハードウェアに依存しないコード最適化を適用することができる．

```
int fib(int x) {
  if (x=0) {
    return 0;
  } else if (x=1) {
    return 1;
  } else {
    return fib(x-2)+fib(x-1);
  }
}

print fib(4);
```

```
  define @fib(x) {
    t1 := x
    t2 := (x=0)
    if not t1 goto l1
    return 0
l1: t3 := (x=1)
    if not t2 goto l2
    return 1
l2: t4 := t1 - 2
    t5 := call fib(t4)
    t6 := t1 - 1
    t7 := call fib(t6)
    t8 := t5 + t7
    return t8
}

define @main() {
    t1 := 4
    t2 := call fib(t1)
    print t2
}
```

図 6.5: 3 番地コードへの変換例

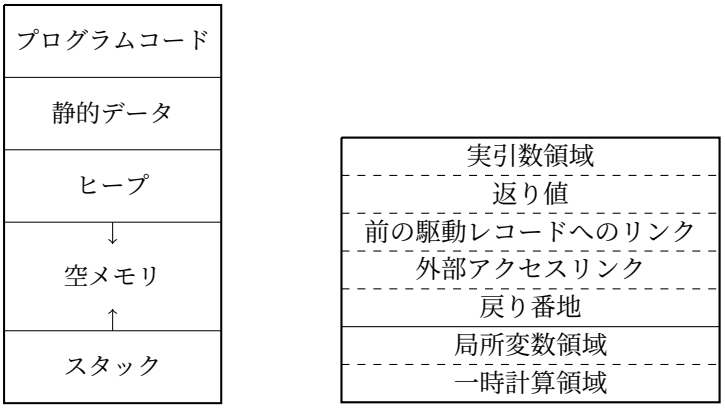
図 6.5 に変換例を示す．ここでは，関数呼び出しとして@関数名　という形をとっている．

6.2　実行環境

プログラミング言語として局所変数を持つことが一般的である．オブジェクト指向言語では，インスタンスが生成されると変数領域を割り当てて実行が終了したとき領域を解放する必要がある．命令型では，関数・手続きが呼び出されたときには，関数・手続きのみで有効な局所変数を割り当てることでプログラムを合成的に記述することが可能になっている．局所変数は関数・手続きの呼び出しごとに動的に割り当てられることで，再帰的な関数定義が可能になっている．関数と手続きの呼び出しは，入れ子構造であることから，**駆動レコード** (Activation record) という単位でスタック構造を操作することで実行環境を実現する．単純に**スタックフレーム** (stack frame) と呼ばれることもある．

6.2.1　駆動レコードの構成

プログラムを実行するプロセスの全体のメモリは図 6.6a のような形となる場合が多い．駆動レコードの構成は一般には図 6.6b のような構成となる．



(a) メモリ配置の例　　(b) 駆動レコードの例

図 6.6: 実行環境のメモリ配置

1. 実引数領域：引数を評価して値を格納して関数・手続きに引き渡す．
2. 返り値：関数の計算が終了して得られる値を格納する．
3. 駆動リンクへのリンク：スタックの格納単位を示す．
4. 外部アクセスリンク：大域変数などへの参照のための番地を格納する．
5. 戻り番地：関数・手続きが終了したときに実行するプログラムカウンタの番地を格納する．
6. 局所変数領域：関数・手続きの実行時に割り当てられる変数領域を示す．
7. 一時計算領域：関数・手続きの実行に必要な一時変数領域を示す．

駆動レコードを単位として，プロセスのメモリ領域に割り当てられる．空きメモリの量はプロセス起動時に決定されることが多い．十分な量を見込んで割り当てられるが，無限ループなどで異常に多くの関数呼び出しが発生するとプロセスのメモリ限界を超える．この場合は，通常，オペレーティングシステムが異常を検出してプロセスが異常終了する．

関数呼び出しとリターン

関数（手続き）の呼び出しは以下の順序で行われる．

1. スタック上に駆動レコードを確保する．
2. 実引数の値を評価して，呼び出しが起こる駆動レコードの領域に書き込む．
3. 呼び出しが発生したプログラムカウンタ PC の値+1 をスタックに格納
4. 前の駆動レコードの先頭のリンクを格納
5. 局所領域をスタック上に確保
6. 呼び出された関数の命令の頭にジャンプする．

関数（手続き）において，リターンが実行されたとき．

1. 関数であれば返し値を駆動レコードの領域に格納
2. 戻りアドレスを駆動レコードから取り出す．
3. 駆動レコードの先頭のリンクを取り出して，現在の駆動レコードを破棄
4. 戻りアドレスへジャンプ

例 6.1. 以下の C 言語風のプログラムに対する実行環境の概要は図**??**のようになる．ここでは，駆動レコードのうちの変数領域のみの状況を示している．Rv は関数の返り値を表す．

```
1  int a;
2
3  int func(int x) {
4    int y;
5    if (x == 1) { return(a);}
6    else {
7      y = func(x-1) * 2;
8    }
9    return (a+y); }
10
11 void main() {
12   int b;
13   b = 3;
14   a = 2;
15   printf("%d\n",func(b)); }
```

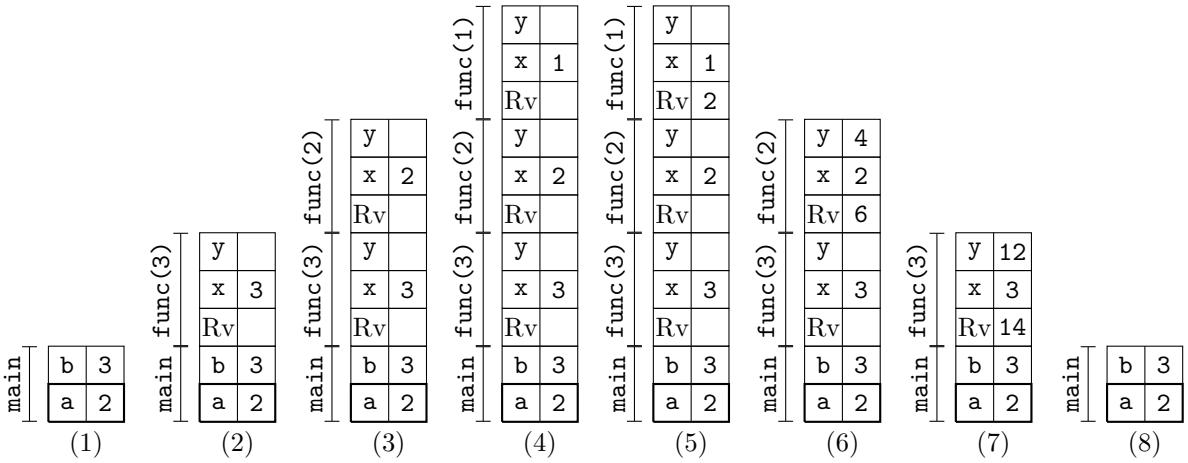


図 6.7: 実行環境の変化

(1) `main` の実行中

(2) `func(3)` の呼び出し直後

(3) `func(3)` から `func(2)` を呼び出した直後

(4) `func(2)` から `func(1)` を呼び出した直後

(5) `func(1)` の返り値を `a` の値として 2 をセット

(6) `func(1)` の駆動レコードを破棄して，`func(2)` の返り値を $2*2+2$ で *6* にセット

(7) `func(2)` の駆動レコードを破棄して，`func(3)` の返り値を $6*2+2$ で *14* にセット

(8) `func(3)` の駆動レコードを破棄.

6.3 構文主導型変換

中間コードは構文解析と同時に生成することができる．このような方法を**構文主導型変換** (Syntax Directed Translation, SDT) と呼ぶ．この変換では生成規則やその途中にコード生成ルーチンを割り当てて，規則の適用にしたがって対応するコード生成ルーチンを呼び出して中間コードを生成する.

6.3.1 構文主導型変換の基礎

コード生成ルーチンとして以下の 3 つの関数を用いる.

- `newTemp()`
  あたらな領域を割り当ててそのメモリ番地を返す．`newTemp()` を実行すると，新たに割り当てられた領域をさすメモリ番地が返り値となる．`x` に新たに領域を割り当てるには，`x.var` := `newTemp()` とする．ここでは，駆動レコード上の局所領域が割り当てられる．関数などの終了によって局所領域が解放されるときに自動的に解放される[1].

- `newLabel()`
  新たなラベルをわりあててそのラベルの命令番地を返す．割り当てられる領域を $L1, L2, \cdots$ で参照する.

- `generate`
  S引数の 3 番地コードを生成して出力に書き込む．各コードとの対応を以下のように定める.

$C = A\theta B$	`generate`(‘C=$A\theta B$’)
$B = \theta A$	`generate`(‘B=θA’)
$B = A$	`generate`(‘B=A’)
if A goto L	`generate`(‘`if` A `goto` L’)
if_not A goto L	`generate`(‘`if_not` A `goto` L’)
goto L	`generate`(‘`goto` L’)
L_i :	`generate`(‘`label` L_i’)

例 6.2. 以下の生成規則を持つ *CF* 文法を考える.

$$
\begin{aligned}
S &\rightarrow \mathsf{ID} = E\ ; \\
E &\rightarrow E^{(1)} + \mathsf{NUM} \\
E &\rightarrow \mathsf{NUM}
\end{aligned}
$$

中間コード生成は，生成規則に生成ルーチンを対応づけることで実現される.

[1] ヒープ領域のような場合は，明示的に解放する必要がある.

生成規則	コード生成ルーチン
$E \to \mathsf{NUM}$	{$E.var$:=newTemp*();*
	generate*(‘$E.var$=*NUM.val*’)*}
$E \to E^{(1)} + \mathsf{NUM}$	{$E.var$:=newTemp*();*
	generate*(‘$E.var$=$E^{(1)}.var$+*NUM.val*’);*}
$S \to \mathsf{ID} = E$;	{generate*(‘*ID.$name$=$E.var$*’)*}

ここで，$E.var$ は，非終端記号 E に対して割り当てられる変数，NUM.val はトークン NUM が表す数値，ID.$name$ は，識別子 ID の名前を表す文字列を表す．

ID=NUM_0+NUM_1+NUM_2; に対する中間コード生成は以下の通りである．

還元系列		生成中間コード
	ID=NUM_0+NUM_1+NUM_2;	$t1 = \mathsf{NUM}_0.val$ *($E_1.var = t1$)*
$\Leftarrow$	ID=E_1+NUM_1+NUM_2;	$t2 = t1$+$\mathsf{NUM}_1.val$ *($E_2.var = t2$)*
$\Leftarrow$	ID=E_2+NUM_2;	$t3 = t2$+$\mathsf{NUM}_2.val$ *($E_3.var = t3$)*
$\Leftarrow$	ID=E_3;	ID.$name = t3$
$\Leftarrow$	S	

実行制御を含む文に対する SDT

条件文のコード生成は以下のようになる．

$$S \to \texttt{if (}\ E\ \texttt{)}\ S^{(1)} \quad \Rightarrow \quad \begin{array}{l} E \text{ のコードを生成 } E.var = t1 \\ \texttt{if not } t1 \texttt{ goto } L1 \\ S^{(1)} \text{ のコードを生成} \\ L1 \end{array}$$

$$S \to \texttt{if (}\ E\ \texttt{)}\ S^{(1)}\ \texttt{else}\ S^{(2)} \quad \Rightarrow \quad \begin{array}{l} E \text{ のコードを生成 } E.var = t1 \\ \texttt{if not } t1 \texttt{ goto } L1 \\ S^{(1)} \text{ のコードを生成} \\ \texttt{goto } L2 \\ L1 : S^{(2)} \text{ のコードを生成} \\ L2 : \end{array}$$

構文解析を生成規則に沿って進めるためには，条件文の生成規則が適用されたときに全体のコードを生成するのではなく，条件式 E が還元されたときに，E のコードを生成して出力する必要がある．

$$\begin{array}{rll} S \to & \texttt{if (}\ E\ \texttt{)} & \{S.label\texttt{:=newLabel}(); \\ & & E \text{ のコード生成 } (E.var \text{ に結果を格納}) \\ & & \texttt{generate}(\text{‘}\texttt{if_not } E.var \texttt{ goto } S.label'); \} \\ & S & S \text{ のコード生成} \\ & & \{\texttt{generate}(\text{‘}\texttt{label } S.label'); \} \end{array}$$

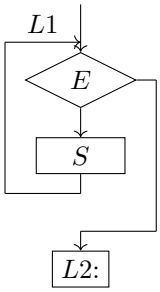
$$
\begin{array}{lll}
S \rightarrow & \texttt{if (}\ E\ \texttt{)} & \{S.label{:=}\texttt{newLabel}();\, S.else{:=}\texttt{newLabel}() \\
 & & E \text{ のコード生成 } (E.var \text{ に結果を格納}) \\
 & & \texttt{generate(`if_not}\ E.var\ \texttt{goto}\ S.else'); \} \\
 & S^{(1)} & S^{(1)} \text{ のコード生成} \\
 & & \{\texttt{generate(`goto}\ S.label'); \} \\
 & \texttt{else} & \{\texttt{generate(`label}\ S.else'); \} \\
 & S^{(2)} & S^{(2)} \text{ のコード生成} \\
 & & \{\texttt{generate(`label}\ S.label'); \}
\end{array}
$$

while 文に対する構文主導型変換は以下の順番に行われる．

$$S \rightarrow \texttt{while (}\ E\ \texttt{)}\ S$$

この生成規則に対する中間コードは以下のようになる．

1： L_1 :
2： E のコード (t1=$E.var$)
3： if_not t1 goto L_2
4： S のコード
5： goto L_1
6： L_2 :



コード生成ルーチンを伴う生成規則は以下のようになる．

$$
\begin{array}{lll}
S \rightarrow & \texttt{while (} & \{S.label1{:=}\texttt{newLabel}(); \\
 & & \texttt{generate(`label}\ S.label1')\} \\
 & E & (E \text{ のコード } (E.var \text{ に結果格納})) \\
 & \texttt{)} & \{S.label2{:=}\texttt{newLabel}(); \\
 & & \texttt{generate(`if_not}\ E.var\ \texttt{goto}\ S.label2') \\
 & S & S \text{ のコード生成} \\
 & & \{\texttt{generate(`goto}\ S.label1'); \\
 & & \texttt{generate(`label}\ S.label2')\}
\end{array}
$$

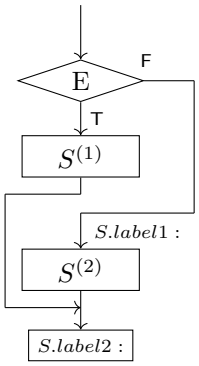
6.3.2 上昇型構文主導型変換

LR パーザーにコード生成ルーチンを組み込む場合，コード生成は還元のタイミングで行う．したがって，条件分岐，ループのように生成規則の途中でコード生成を行う必要がある場合には，新たな非終端記号を導入すれば還元の際にのみコード生成ルーチンを呼び出すことができる．

$$S \to \texttt{if (}\ E\ \texttt{)}\ S^{(1)}\ \texttt{else}\ S^{(2)}$$

についてはフローチャートに沿って以下のような生成ルーチンを付加することになる．

$$
\begin{array}{lcll}
S & \to & \texttt{if (}\ E\ \texttt{)} & \{S.label1 = \texttt{newLabel}(); \\
 & & & \texttt{generate}(\text{`}\texttt{if_not}\ E.var\ \texttt{goto}\ S.label1') \\
 & & S^{(1)} & \{S.label2 = \texttt{newLabel}(); \\
 & & & \texttt{generate}(\text{`}\texttt{goto}\ S.label2'); \\
 & & & \texttt{generate}(\text{`}\texttt{label}\ S.label1')\} \\
 & & \texttt{else}\ S^{(2)} & \{\texttt{generate}(\text{`}\texttt{label}\ S.label2')\}
\end{array}
$$



この生成規則に対して，コード生成が規則を還元する場合にのみ発生する等価な生成規則を以下のように構成する．

$$\{I \to \texttt{if (}\ E\ \texttt{)},\quad S' \to IS^{(1)},\quad S \to S'\ \texttt{else}\ S^{(2)}\}$$

$$
\begin{array}{lcll}
I & \to & \texttt{if (}\ E\ \texttt{)} & \{I.label = \texttt{newLabel}(); \\
 & & & \texttt{generate}(\text{`}\texttt{if_not}\ E.var\ \texttt{goto}\ I.label')\} \\
S' & \to & IS^{(1)} & \{S'.label = \texttt{newLabel}(); \\
 & & & \texttt{generate}(\text{`}\texttt{goto}\ S'.label'); \\
 & & & \texttt{generate}(\text{`}\texttt{label}\ I.label')\} \\
S & \to & S'\ \texttt{else}\ S^{(2)} & \{\texttt{generate}(\text{`}\texttt{label}\ S'.label')\}
\end{array}
$$

`while` 文について以下のような生成規則を構成する．

$$\{I \to \texttt{while},\quad S' \to \texttt{(}\ E\ \texttt{)},\quad S \to IS'S^{(1)}\}$$

$$
\begin{array}{lcll}
I & \to & \texttt{while} & \{I.label = \texttt{newLabel}(); \\
 & & & \texttt{generate}(\text{`}\texttt{label}\ I.label')\} \\
S' & \to & \texttt{(}\ E\ \texttt{)} & \{S'.label = \texttt{newLabel}(); \\
 & & & \texttt{generate}(\text{`}\texttt{if_not}\ E.var\ \texttt{goto}\ S'.label')\} \\
S & \to & IS'S^{(1)} & \{\texttt{generate}(\text{`}\texttt{goto}\ I.label'); \\
 & & & \texttt{generate}(\text{`}\texttt{label}\ S'.label')\}
\end{array}
$$

6.3.3 下降型構文主導型変換

LL(1) 下降型構文解析にコード生成に組み込む場合は，より直観的にコード生成することができる．生成規則の右辺で生成ルーチンに到達すれば，そのルーチンを実行すればよい．このことは下降型の構文解析では，構文図を用いて示される．

構文図をたどる際に構文要素の項目に応じてコードを生成する．

例 6.3. 以下の拡張 *BNF* で定義されるプログラム P を考える．

$$
\begin{array}{lcl}
P & := & S\ (;S)^* \\
S & := & \mathsf{ID} := E \mid \texttt{if}B\ \texttt{then}\ S\ \texttt{else}S\ \texttt{fi} \mid \texttt{while}\ B\ \texttt{do}\ S\ \texttt{od} \\
E & := & T((+|-)T)^* \\
T & := & F((*|/)F)^* \\
F & := & (\ E\ ) \mid \mathsf{ID} \mid \mathsf{NUM} \\
B & := & A(||A)^* \\
A & := & N(\&\&N)^* \\
N & := & C \mid !C \\
C & := & (\ B) \mid \mathsf{ID} < \mathsf{ID} \mid \mathsf{ID} < \mathsf{NUM} \mid \mathsf{NUM} < \mathsf{ID}
\end{array}
$$

これらの構文図式は図 *6.8* のようになる．

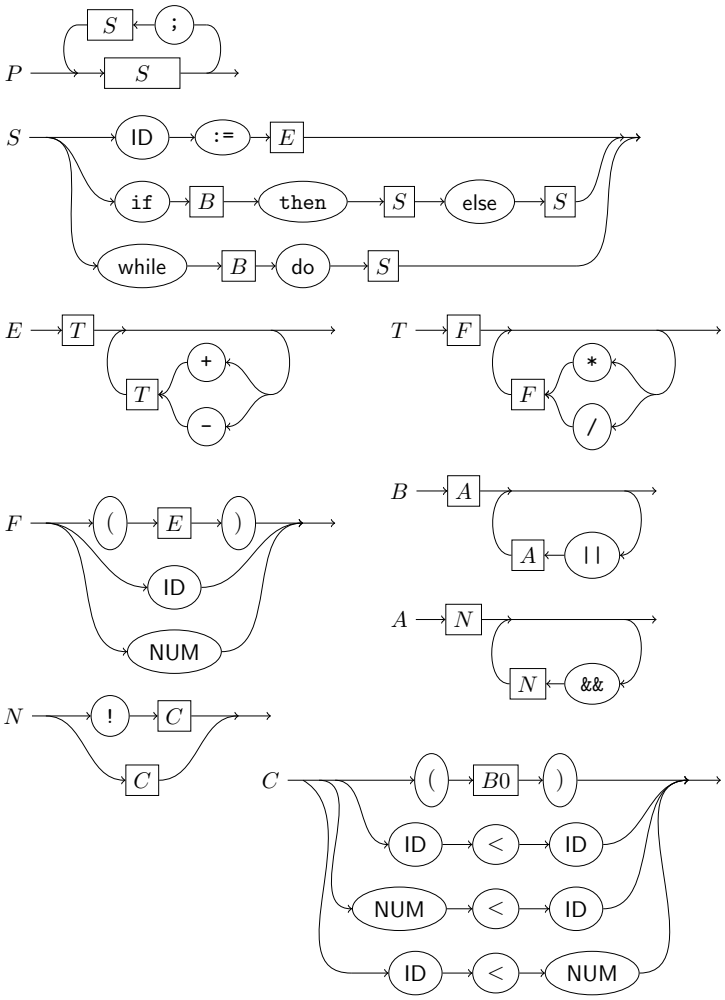


図 6.8: P の構文図式

例えば，S については，解析の経路をたどりながら以下のように中間コードを生成する．nextsym に先読みトークンの情報が格納されており，getNextsym() で次のトークンを読む．

```
// nextsym には先読みトークンがあらかじめ格納されている
If nexsym.kind = ID then                          // 代入文
    getNextsym()
    If nextsym.kind = '=' then                    // 代入記号
        E のコードを生成 ($tempE に結果を格納)      // E
        generate('var(ID.val) = $tempE')          // 代入本体
    else error()
else if nextsym.kind = if then                    // 条件文
    getNextsym()
```

```
    B のコードを生成 (tempB に結果を格納)        // B
    L1 = newlabel()                              // 偽の場合の飛び先
    generate('if_not tempB goto L1')             // 分岐命令本体
    S のコードを生成                             // 条件が真の場合の S
    L2 = newlabel()                              // 偽の命令の後への飛び先
    generate('goto L2')
    generate('label L1')
    S のコードを生成                             // 条件が偽の場合の S
    generate('label L2')                         // 真の S のジャンプ先
else if nextsym.kind = while then                // 繰り返し文
    getNextsym()
    L1 = newlabel()
    generate('label L1')                         // 繰り返しで戻るラベル
    B のコードを生成 (tempB に結果を格納)        // B
    If nextSym.kind = do then
        L2 = newlabel()
        generate('if_not tempB goto L2')
        S のコードを生成                         // 繰り返し本体の S
        generate('goto L1')                      // 繰り返しに戻る
        generate('label L2')                     // 脱出先のラベル
    else error()
else
    error()
```

ここで，nextsym.$kind$ はトークンの種類，nextsym.val はトークンの値を表す．var(s) は文字列 s を持つ識別子が変数であるとき，記号表を参照してメモリ内の番地を与える．

他の非終端記号の構文グラフについても同様に 3 番地コードを生成することができる．

6.3.4 構文主導定義による意味定義

構文主導定義 (Syntax-Directed Translation, SDD) は生成規則の構文要素に「属性」を割り当てることによって，特定の意味を与えることができる．構文主導定義では，コード生成ルーチンを陽に与える代わりに，構文要素間に成立する等式を意味規則として与える．形式言語理論における属性文法 (Attributed Grammar) に基づく．非終端記号 X に対して $X.a$ は，X に対する属性 a を表す．例えば，生成規則 $E \to E^{(1)}+\mathsf{NUM}$ に対して，各構文要素がもつ値属性 (val) に対する関係 $E.val = E^{(1)}.val + \mathsf{NUM}.val$ によってこの生成規則が与える意味を定義する．属性は，数，型，表参照，文字列など，どういった種類の値でもよい．

例 6.4. 以下の意味規則つきの生成規則を持つ文法 $G_A = (\{L, E, T, F\}, \{+, *, (,), \mathsf{NUM}, \mathbf{n}\}, P, L)$

を考える．

$$P = \left\{ \begin{array}{lll} 0: & L \to E\ \mathbf{n} & \{L.val = E.val\} \\ 1: & E \to E_1 + T & \{E.val = E_1.val + T.val\} \\ 2: & E \to T & \{E.val = T.val\} \\ 3: & T \to T_1 * F & \{T.val = T_1.val \times F.val\} \\ 4: & T \to F & \{T.val = F.val\} \\ 5: & F \to (E) & \{F.val = E.val\} \\ 6: & F \to \mathsf{NUM} & \{F.val = \mathsf{NUM}.val\} \end{array} \right\}$$

規則 $0 : L \to E\ \mathbf{n}$ は，計算終了記号 $\mathbf{n}$ を入力することで，値計算をすることを表す．NUM は数のトークンであり，NUM.val は，そのトークンが表す属性としての値である．図 *6.9* は $3 + 4 * 5$ に対する構文解析木とその属性評価である．

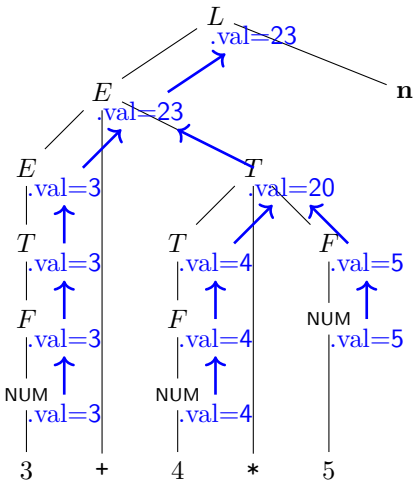


図 6.9: 解析木と属性評価

定義 6.1. 文脈自由文法 $G = (N, \Sigma, P, S)$ に対する *SDD* は $(G, \mathcal{A}, E)$ として与えられる．

- $\mathcal{A}$ は属性の集合である．$X \in N \cup \Sigma$ に対して，$\mathcal{A}|_X$ は，X に割り当てられた属性の集合を表す．$X.a$ は，X に割り当てられた属性値を表し，$X.a \in \mathcal{A}$ である．
- E は生成規則 $p \in P$ に対して，意味規則を与える．意味規則は，属性間に成立する等式の集合である．属性間に成立する等式は以下の形を持つ．

  $$X_{i_1}.a_{j_1} = f(X_{i_2}.a_{j_2}, \cdots, X_{i_n}.a_{j_m}) \in E(p)$$

  であるとき，X_k は p に現れる $N \cup \Sigma$ である．f は意味関数であり，引数となる属性に対して定義されているとする．f は引数となる属性に対して計算可能な関数であり，計算手続きはメモリ更新などの副作用を持たない．
- 意味規則の等式において，X_{i_1} が p の左辺の非終点記号であるとき，$X_{i_1}.a_{j_1}$ を合成属性 *(syntheiszed attribute)* とよび，そうでないとき相続属性 *(inherited attribute)* とよぶ．$X \in$

$N \cup \Sigma$ に対して，合成属性の集合を $S(X)$, 相続属性の集合を $I(X)$ と書く．$X \in \Sigma$ に対して $I(X) = \varnothing$ であり，すべての属性 $X.a$ は，字句解析時に決定している．

E の定義において，以下の 3 つの条件を満たす．

1. $S(X) \cap I(X) = \varnothing$
2. $S(X) \cup I(X) = \mathcal{A}|_X$
3. $Y_1 \to Z_1, \cdots, Z_n \in P$ に対して，$Y.a \in S(Y)$ および $Z_i.a \in I(Z_i)$ のそれぞれに対して，ちょうど１つの意味規則の等式が存在する．

例 6.5. 次に与えられる属性文法について考える．

$G = (\{T, L\}, \{\mathsf{max}, \mathsf{min}, \mathsf{NUM}, `(', `)', `,'\}, P, T)$ $\mathcal{A} = \{val, op\}$,

$$P, E = \left\{ \begin{array}{ll} T \to \mathsf{min}`(' L `)' & \{L.op = \mathsf{min}, T.val = L.val\} \\ T \to \mathsf{max}`(' L `)' & \{L.op = \mathsf{max}, T.val = L.val\} \\ L \to \mathsf{NUM} & \{L.val = \mathsf{NUM}.val\} \\ L \to L^{(1)} `,' \mathsf{NUM} & \{L.val = f(L.op, L^{(1)}.val, \mathsf{NUM}.val), L^{(1)}.op = L.op\} \end{array} \right\}$$

$f(L.op, L^{(1)}.val, \mathsf{NUM}.val)$ は，$L.op(L^{(1)}.val, \mathsf{NUM}.val)$ を計算する意味関数とする．

$S(T) = \{val\}$, $I(T) = \varnothing$, $S(L) = \{val\}$, $I(L) = \{op\}$

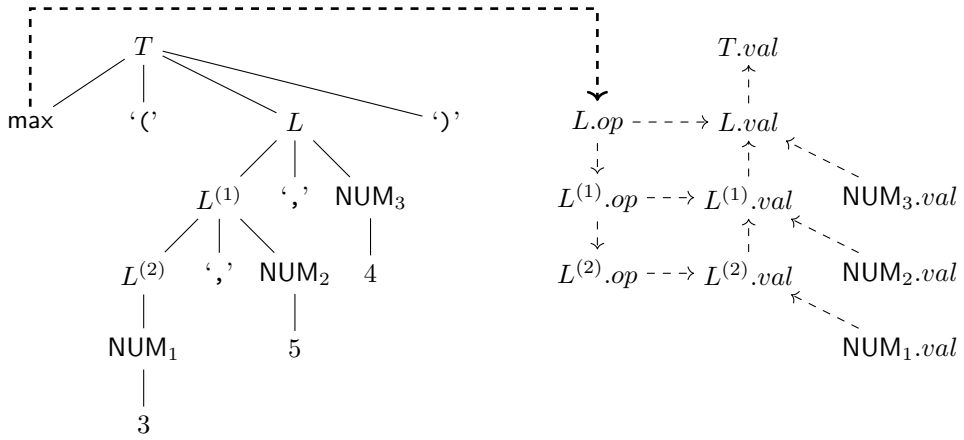


図 6.10: 属性の評価

属性文法において，属性の計算方法が巡回すると属性の計算ができなくなる．例えば，以下の意味規則を持つ場合は属性の計算ができない．

$$A \to B \quad \{A.s = B.i, B.i = A.s + 1\}$$

属性文法に基づく意味を計算する場合は，合成属性に限定する (S-Attribute Definition)，あるいは，相続属性の出現を限定する (L-Attribute Definition) ことによって巡回を避けるための十分条件が知られている．

6.3.5　コンパイラコンパイラによるコード生成

ソースプログラムの仕様

ブロック構造とループ構造を持つ単純なプログラミング言語 PL-Simple を以下のように定義する．

⟨program⟩::='program' 'IDENT' ';' ⟨outblock⟩ '.'
⟨outblock⟩::=⟨var_decl_part⟩ ⟨statement⟩
⟨var_decl_part⟩::=⟨var_decl_list⟩';'|/* empty */
⟨var_decl_list⟩::=⟨var_decl_list⟩';'⟨var_decl⟩|⟨var_decl⟩
⟨var_decl⟩::='var'⟨id_list⟩
⟨id_list⟩::='IDENT'|⟨id_list⟩','IDENT'
⟨statement_list⟩::=⟨statement_list⟩';'⟨statement⟩|⟨statement⟩
⟨statement⟩::=⟨assignment_statement⟩|⟨if_statement⟩
|⟨while_statement⟩|⟨for_statement⟩|⟨null_statement⟩
|⟨block_statement⟩|⟨read_statement⟩|⟨write_statement⟩
⟨assignment_statement⟩::='IDENT':='⟨expression⟩
⟨if_statement⟩::='if'⟨condition⟩'then'⟨statement⟩⟨else_statement⟩
⟨else_statement⟩::='else'⟨statement⟩|/ ∗ *empty* ∗ /
⟨while_statement⟩::='while'⟨condition⟩'do'⟨statement⟩
⟨for_statement⟩::='for''IDENT':='⟨expression⟩'to'⟨expression⟩'do'⟨statement⟩
⟨block_statement⟩::='begin'⟨statement_list⟩'end'
⟨read_statement⟩::='read''('IDENT')'
⟨write_statement⟩::='write''('⟨expression⟩')'
⟨null_statement⟩::=/* empty */
⟨condition⟩::=⟨expression⟩'='⟨expression⟩|⟨expression⟩'<>'⟨expression⟩
|⟨expression⟩'<'⟨expression⟩|⟨expression⟩'<='⟨expression⟩
|⟨expression⟩'>'⟨expression⟩|⟨expression⟩'>='⟨expression⟩
⟨expression⟩::=⟨term⟩|'+'⟨term⟩|'-'⟨term⟩|⟨expression⟩'+'⟨term⟩|⟨expression⟩'-'⟨term⟩
⟨term⟩::=⟨factor⟩|⟨term⟩'*'⟨factor⟩|⟨term⟩'div'⟨factor⟩
⟨factor⟩::=⟨var_name⟩|'NUMBER'|'('⟨expression⟩')'
⟨var_name⟩::='IDENT'

ここで，識別子トークン IDENT は [a-zA-Z][a-zA-Z_0-9]*, 数値トークン NUMBER は (+-—)?[0-9]+—で定義されるとする．ここで?は 0 回または 1 回にマッチする文字列，+は 1 回以上の繰り返

しにマッチする文字列を表す.

目的コードの仕様

生成の対象とする目的コードセットを以下のようなものとする．s は実行時に用いられるデータを格納するための配列であり十分な領域が確保されているとする．s は評価においてはスタックとして用いられる．

命令長は 1 word であり、命令は次の形式を持つ。

OPCODE	b	i	a

b ベースレジスタの指定。

i インデックスレジスタの指定。

a アドレス（displacement)。命令により定数や演算の指定にも使う。

命令領域（読出し専用）は word 単位で番地を持ち、0 番地から実行する。PC が示す番地の命令を IR に格納し、PC を 1 増加させてから、IR の命令を実行する。初期状態は次のとおりである。

$$\mathrm{PC} = 0, \quad R_1 = 0, \quad R_2 = -1, \quad R_3 = 0, \quad R_4 = 0.$$

以下では動作を C 言語風に記述する。スタックは整数データを読み書きする領域であり、`s[Rn]` はレジスタ `Rn` が示す番地の内容を表す。各動作は記述順に実行する。

1. Load literal

`lit`	b	i	a

```
R2 = R2 + 1; s[R2] = a
```

b と i は無視される。 原文の`'a'` は即値 a を表す。

2. Load

`lod`	b	i	a

```
R2 = R2 + 1; s[R2] = s[Rb + Ri + a]
```

b と i が指定するレジスタの内容と a を加えたものをオペランドアドレスとする。ただし、$b = 0$ のときベースによるアドレス修飾は行わず、$i = 0$ のときも同様である。

3. Store

`sto`	b	i	a

```
s[Rb + Ri + a] = s[R2]; R2 = R2 - 1
```

アドレスの計算は `lod` と同様であり、$b = 0$ または $i = 0$ の項は加えない。

4. Move to index register

`mvx`	b	i	a

```
Ri = s[R2]; R2 = R2 - 1
```

b と a は無視される。

5. Arithmetic operation

opr	b	i	a

a により次の 11 種類の演算を指定する。

b と i は無視される。

a	動作
0	`s[R2] = -s[R2]`
1	`R2 = R2 - 1; s[R2] = s[R2] + s[R2+1]`
2	`R2 = R2 - 1; s[R2] = s[R2] - s[R2+1]`
3	`R2 = R2 - 1; s[R2] = s[R2] * s[R2+1]`
4	`R2 = R2 - 1; s[R2] = s[R2] div s[R2+1]`
5	`R2 = R2 - 1;` $s[R_2] = s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。
6	`R2 = R2 - 1;` $s[R_2] \neq s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。
7	`R2 = R2 - 1;` $s[R_2] < s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。
8	`R2 = R2 - 1;` $s[R_2] \leq s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。
9	`R2 = R2 - 1;` $s[R_2] > s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。
10	`R2 = R2 - 1;` $s[R_2] \geq s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。

6. Unconditional Jump

jmp	b	i	a

```
PC = a
```

b と i は無視される。

7. Jump on condition

jpc	b	i	a

```
if s[R2] = 0 then PC = a;
R2 = R2 - 1
```

b と i は無視される。

8. Read data

get	b	i	a

`R2 = R2 + 1` として、標準入力から次のデータ（整数値）を読み、`s[R2]` に格納する。

b、i、a は無視される。

9. Write data

put	b	i	a

`s[R2]` の内容を標準出力に出力し、`R2 = R2 - 1` とする。

b、i、a は無視される。

JAVACC によるコード生成

`newlabel()` はラベルの記号を生成して表に登録し，表のエントリーポイントを返す．`genlabel(label)` は `label` のエントリーに対して，次に生成される命令へのポインタを格納する．

`while` 文については以下のようにコード生成ルーチンを生成規則に挿入する．

```
void WhileStatement() : { Label startlabel; Label exitlabel; } {
  <WHILE> { startlabel = newlabel(); genlabel(startlabel); }
  Condition() { exitlabel = newlabel(); emit("jpc", exitlabel); }
  <DO> Statement() { emit("jmp", startlabel); genlabel(exitlabel); }
}
```

if 文については，以下のように生成規則を拡張する．LOOKHEAD は，else が次にあるかどうかを先読みする．

```
void IfStatement() : { Label falselabel; } {
  <IF> Condition() { falselabel = newlabel(); emit("jpc", falselabel); }
  <THEN> Statement() ElseStatement(falselabel)
}
void ElseStatement(Label falselabel) : { Label endlabel = null; } {
  [ LOOKAHEAD(<ELSE>) <ELSE>
    { endlabel = newlabel(); emit("jmp", endlabel); genlabel(falselabel); }
    Statement() ]
  { if (endlabel != null) genlabel(endlabel); else genlabel(falselabel); }
}
```

式については以下のようにコード生成する．BNF が左再帰製をもつため，$_{tail}$ をもつ非終端記号を導入する．

```
void Expression() : {} {
  ( Term() | <PLUS> Term() #UnaryPlus(1)
  | <MINUS> Term() #Negate(1) { emit("opr", 0); } )
  Expression_tail()
}
/* <expression_{tail}>. Emit before recursion to preserve left associativity. */
void Expression_tail() #void : {} {
  [ ( <PLUS> Term() #Add(2) { emit("opr", 1); }
    | <MINUS> Term() #Subtract(2) { emit("opr", 2); } )
    Expression_tail() ]
}
void Term() : {} { Factor() Term_tail() }
/* <term_{tail}> */
void Term_tail() #void : {} {
  [ ( <TIMES> Factor() #Multiply(2) { emit("opr", 3); }
    | <DIV> Factor() #Divide(2) { emit("opr", 4); } )
    Term_tail() ]
}
void Factor() : {} { VarName() | Number() | <LPAREN> Expression() <RPAREN> }
```

```
void VarName() : { Token name; } { name=Identifier() { load(variable(name)); } }
Token Identifier() : { Token token; } {
  token=<IDENT> { jjtThis.jjtSetValue(token.image); return token; }
}
```

フルセットの javacc 記述を `https://github.com/syuen1/26compiler-support/PLsimple-javacc` で公開する．

SLY によるコード生成

SLY を用いてコード生成する場合，LALR(1) による構文解析によるので，還元時にコード生成する．コード生成規則を生成のタイミングで以下のように分割する．

⟨assignment_target⟩::='IDENT''ASSIGN'

⟨assignment_statement⟩::=⟨assignment_target⟩⟨expression⟩

```
@_("IDENT ASSIGN")
def assignment_target(self, p):
    return self.variable(p.IDENT)

@_("assignment_target expression")
def assignment_statement(self, p):
    self.emit("sto", p.assignment_target)
```

⟨if_head⟩::='IF'⟨condition⟩'THEN'

⟨if_statement⟩::=⟨if_head⟩⟨statement⟩⟨else_statement⟩

⟨else_statement⟩::='ELSE'⟨else_start⟩⟨statement⟩|/* empty */

⟨else_start⟩::=/* empty */

```
@_("IF condition THEN")
def if_head(self, p):
    return self.emit("jpc")
    label = self.newlabel()
    self.emit("jpc", label)
    return label

@_("if_head statement else_statement")
```

```
    def if_statement(self, p):
        # An absent else patches the false jump; a present else patches its end jump.
        self.patch(p.if_head if p.else_statement is None else p.else_statement)
        # Define the false label for an absent else, or the end label for a present else.
        self.genlabel(p.if_head if p.else_statement is None else p.else_statement)

    @_("ELSE else_start statement")
    def else_statement(self, p):
        return p.else_start

    @_("%prec IF_WITHOUT_ELSE")
    def else_statement(self, p):
        return None

    @_("")
    def else_start(self, p):
        # Stack immediately to the left: if_head, statement, ELSE.
        false_jump = p[-3]
        end_jump = self.emit("jmp")
        self.patch(false_jump)
        end_jump = self.newlabel()
        self.emit("jmp", end_jump)
        self.genlabel(false_jump)
        return end_jump
```

⟨loop_start⟩::=/* empty */

⟨while_head⟩::='WHILE'⟨loop_start⟩⟨condition⟩'DO'

⟨while_statement⟩::=⟨while_head⟩⟨statement⟩

```
    @_("")
    def loop_start(self, p):
        return len(self.code)
        label = self.newlabel()
        self.genlabel(label)
        return label

    @_("WHILE loop_start condition DO")
    def while_head(self, p):
```

```
        exit_label = self.newlabel()
        self.emit("jpc", exit_label)
        return WhileContext(p.loop_start, self.emit("jpc"))
        return WhileContext(p.loop_start, exit_label)

    @_("while_head statement")
    def while_statement(self, p):
        context = p.while_head
        self.emit("jmp", context.start)
        self.patch(context.exit_jump)
        self.genlabel(context.exit_jump)
```

還元時にコード生成を行う場合は，上記のようにコード生成をするタイミングで ε 規則を追加することによって，プログラミング言語を定義する文法の生成規則を拡張してコード生成を行うことができる． if 文においては，条件のコードを生成した後，条件が偽であった場合の飛び先命令 jpc を生成し，else の前で then で実行した命令が終了後の jmp 命令を ε 規則で生成し，else の後で条件が偽であった場合の飛び先番地を決定している．while 文では，ループ条件の前にループ本体の実行後のジャンプ先ラベルを生成し，条件の直後にループの脱出条件をチェックする命令 jpc を生成している．

フルセットの sly 記述を https://github.com/syuen1/26compiler-support/PLsimple-sly で公開する．

6.4 コード生成例

```
program PL0A;
var n, sum;
begin
  n := 10;
  sum := 0;
  while n > 0 do
  begin
    sum := sum + n;
    n := n - 1;
  end;
  write(sum)
end.
```

PL0A.pl0 (1 から 10 までの和を表示)

```
1  int,0,0,2
2  lit,0,0,10
3  sto,0,0,0
4  lit,0,0,0
5  sto,0,0,1
6  lod,0,0,0
7  lit,0,0,0
8  opr,0,0,9
9  jpc,0,0,18
10 lod,0,0,1
11 lod,0,0,0
12 opr,0,0,1
13 sto,0,0,1
14 lod,0,0,0
15 lit,0,0,1
16 opr,0,0,2
17 sto,0,0,0
18 jmp,0,0,5
19 lod,0,0,1
20 put,0,0,0
```

```
program PL0B;
var m, n, x;
begin
  read(n);
  while 1 < n do
    begin
      x := n;
      begin
        m := x div 2;
        while x <> (x div m) * m do
          m := m - 1;
        if m = 1 then
          write(x)
      end;
      n := n - 1
    end
end.
```

PL0B.pl0 (入力した値より小さい素数を出力)

```
1  int,0,0,3
2  get,0,0,0
3  sto,0,0,1
4  lit,0,0,1
5  lod,0,0,1
6  opr,0,0,7
7  jpc,0,0,37
8  lod,0,0,1
9  sto,0,0,2
10 lod,0,0,2
11 lit,0,0,2
12 opr,0,0,4
13 sto,0,0,0
14 lod,0,0,2
15 lod,0,0,2
16 lod,0,0,0
17 opr,0,0,4
18 lod,0,0,0
19 opr,0,0,3
20 opr,0,0,6
21 jpc,0,0,26
22 lod,0,0,0
23 lit,0,0,1
24 opr,0,0,2
25 sto,0,0,0
26 jmp,0,0,13
27 lod,0,0,0
28 lit,0,0,1
29 opr,0,0,5
30 jpc,0,0,32
31 lod,0,0,2
32 put,0,0,0
33 lod,0,0,1
34 lit,0,0,1
35 opr,0,0,2
36 sto,0,0,1
37 jmp,0,0,3
```